PROJECT 1: THREADS DESIGN DOCUMENT

Team member	First Name	Last Name	Email
1	Indumathi	Thamizharasan	ithamizh@buffalo.edu
2	Rishitha	Saravanan Priya	rsaravan@buffalo.edu
3	Aishwarya	Eshwar	aeshwar@buffalo.edu

---- PRELIMINARIES ----

- Implemented all required functionalities for thread scheduling, including modifications to thread.c and synch.c for priority scheduling and proper handling of sleep/wake-up using timer_sleep.
- We also worked on the extra credit functionalities, such as handling nested priority donation and ensuring the MLFQ scheduler behaves as expected under various conditions.
- Throughout the project, we emphasized writing efficient and modular code, especially in parts involving interrupts and scheduling logic, to maintain performance and clarity.

References:

- Pintos documentation
- Source code
- https://ublearns.buffalo.edu/d2l/le/content/267253/Home
- Recitations

ALARM CLOCK
==========

---- DATA STRUCTURES ----

A1: Copy here the declaration of each new or changed `struct' or `struct' member, global or static variable, `typedef', or enumeration. Identify the purpose of each in 25 words or less.

```
struct thread {
  ...
  int64_t wkup_ticks;
  ...
};
static struct list sleep_list;
```

- `wkup_tick`: Stores the tick count when the thread should wake up.

- `sleep_list`: A global list holding threads that are currently sleeping, sorted by wake-up time.

---- ALGORITHMS ----

A2: Briefly describe what happens in a call to timer_sleep(), including the effects of the timer interrupt handler.

The function timer_sleep(ticks) figures out when a thread should wake up and adds it to the sleep_list in the right order using list_insert_ordered(). Every time there's a timer interrupt, thread_tick() goes through the sleep_list and wakes up any threads that are ready to run again.

A3: What steps are taken to minimize the amount of time spent in the timer interrupt handler?

To reduce the duration spent in the timer interrupt, the work of waking sleeping threads is done inside thread_tick(), which is invoked by timer_interrupt(). The sleep_list is kept sorted by wakeup time, so scanning from the front and stopping early is sufficient, meaning a complete list traversal is not necessary.

---- SYNCHRONIZATION ----

A4: How are race conditions avoided when multiple threads call timer_sleep() simultaneously?

To prevent race conditions, we use intr_disable() and intr_set_level() to turn off interrupts while we access or modify the sleep_list. This way, we ensure that there's mutual exclusion during the timer_sleep() function.

A5: How are race conditions avoided when a timer interrupt occurs during a call to timer_sleep()?

When accessing the sleep_list in both timer_sleep() and the interrupt handler, interrupts are turned off. This ensures that there are no simultaneous changes happening, which helps keep the list intact and in sync.

---- RATIONALE ----

A6: Why did you choose this design? In what ways is it superior to another design you considered?

The design is simple and efficient, making great use of Pintos's built-in thread blocking mechanism. By utilizing a sorted list, it keeps the overhead in the interrupt handler to a minimum and cuts down on unnecessary looping. This approach is definitely better than busy-waiting, which just ends up wasting valuable CPU cycles.

PRIORITY SCHEDULING
==========

---- DATA STRUCTURES ----

B1: Copy here the declaration of each new or changed `struct' or `struct' member, global or static variable, `typedef', or enumeration. Identify the purpose of each in 25 words or less.

```
/* In thread.c: */
static struct list ready_list;  // List of ready threads.

/* In struct thread (thread.h): */
int priority;                   // Current priority.
int initial_priority;           // Base priority (before donation).
struct list hold_lock_list;     // Locks held by thread.
struct lock *block_lock;        // Lock currently waiting on.

/* In struct lock (synch.h): */
int lock_priority;              // Max priority among waiters.
struct list_elem elem_lock;     // For thread's lock list.
```

B2: Explain the data structure used to track priority donation. Use ASCII art to diagram a nested donation.

We keep tabs on donations through the hold_lock_list found in the struct thread, which keeps a record of all the locks that the thread currently holds. Each lock has a lock_priority that gets updated whenever a donation occurs.

For example, Thread A, which has a priority of 10, is holding Lock X, while Thread B, with a priority of 20, is waiting for Lock X.
Thread A (holds Lock X) (priority=10)
Thread B (waits on Lock X) (priority=20)

A <- block_lock = X // A is blocking Lock X

X <= lock_priority = 20 // Lock X has a lock_priority of 20

hold_lock_list (A) = [X] // The hold_lock_list for Thread A contains [X]

Donation Diagram:

```
Thread A (P=10)
        |
Thread B (P=20)
        |
Thread A's P updated to 20
        |
Thread C (P=30)
        |
Lock X (lock_priority = 30)
        |
Thread A's P updated to 30
```

---- ALGORITHMS ----

B3: How do you ensure that the highest priority thread waiting for a lock, semaphore, or condition variable wakes up first?

We sort ready_list, semaphore.waiters, and cond.waiters using thread_compare_priority_order(). This way, we make sure that the highest-priority thread gets selected first.

B4: Describe the sequence of events when a call to lock_acquire() causes a priority donation. How is nested donation handled?

As a thread invokes lock_acquire(), if the thread holding the lock, it sets block_lock and does priority donation propagation in depth. Each lock modifies its lock_priority and the holder's priority is renewed through thread_update_priority().

B5: Describe the sequence of events when lock_release() is called on a lock that a higher-priority thread is waiting for.

In the function lock_release(), we take the lock out of the hold_lock_list, clear the lock's holder, and call thread_update_priority() to get the thread back to its correct priority after letting go of the donated lock.

---- SYNCHRONIZATION ----

B6: Describe a potential race in thread_set_priority() and explain how your implementation avoids it. Can you use a lock to avoid this race?

In the function thread_set_priority(), we turn off interrupts to prevent any race conditions that might occur when we're adjusting the base priority while locks are still in play. Using a lock helps us avoid circular dependencies here, so just disabling interrupts does the trick.

---- RATIONALE ----

B7: Why did you choose this design? In what ways is it superior to another design you considered?

This design makes it super easy to track donations by using hold_lock_list, and it supports nesting in a really intuitive way. Plus, it’s much simpler to debug compared to other options that rely on complicated dependency graphs or extra structures.

ADVANCED SCHEDULER
==========

---- DATA STRUCTURES ----

C1: Copy here the declaration of each new or changed `struct' or `struct' member, global or static variable, `typedef', or enumeration. Identify the purpose of each in 25 words or less.

```
/* In struct thread */
int nice;                // Niceness value: -20 to 20
fixed_point_t recent_cpu;        // Recent CPU usage as a fixed-point number

/* Global variables */
fixed_point_t LoadAvg;          // System load average
```

---- ALGORITHMS ----

C2: Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a recent_cpu value of 0. Fill in the table below showing the scheduling decision and the priority and recent_cpu values for each thread after each given number of timer ticks:

- Recent_cpu: This tracks how much CPU time a thread has used and gets updated periodically.

  Recent_cpu = recent_cpu + 1

- Priority : Thread's priority is given by its recent_cpu.

  Priority = base_priority - (recent_cpu / 4)

Timer	recent_cpu			priority			thread
ticks	A	B	C	A	B	C	Thread to run
0	0	0	0	63	61	59	A
4	1	0	0	63	61	59	A
8	2	0	0	62	61	59	A
12	3	0	0	61	61	59	A
16	4	0	0	61	61	59	A
20	5	1	0	60	60	59	A
24	6	2	1	59	59	58	A
28	7	3	2	58	58	57	A
32	8	4	3	57	57	56	A
36	9	5	4	56	56	55	A

C3: Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

Yes. There's some ambiguity in how rounding is approached. For our priority calculations, we went with fix_round(), and for addition, multiplication, and division, we used fix_add/mul/div to stick closely to Pintos conventions. Our scheduler aligns with this approach.

C4: How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

When calculating recent_cpu and LoadAvg in an interrupt context, it can really slow down the system. To tackle this, we've decided to offload those updates so they only happen every few ticks, which helps boost overall performance.

---- RATIONALE ----

C5: Briefly critique your design, pointing out advantages and disadvantages in your design choices. If you were to have extra time to work on this part of the project, how might you choose to refine or improve your design?

Our design keeps the scheduling logic separate from the core thread logic, which really boosts modularity. We also used macros to simplify fixed-point arithmetic for better clarity. If we had a bit more time, we'd definitely work on enhancing donation rollback and fine-tuning the condition variable logic.

C6: The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

We used existing function-based fixed-point arithmetic libraries to avoid arithmetic errors and improve code clarity. By defining an abstract data type, fixed_point_t, and implementing functions like fix_add, fix_sub, fix_mul, and fix_div, we encapsulated the conversion and scaling logic for fixed-point numbers in one place. This abstraction minimized bugs caused by manual arithmetic and made our scheduling code more readable and maintainable. It also ensured consistent behavior across all components that use fixed-point math, particularly in MLFQS calculations.

SURVEY QUESTIONS
==========

1. In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

As a group, we found the assignment challenging but very rewarding. The Alarm Clock part was manageable once we got the hang of thread blocking and the timer interrupt, but Priority Scheduling and the MLFQS definitely pushed us more. Debugging priority donation and getting the fixed-point math just right took longer than expected. Overall, it wasn't too hard, but it was time-consuming, mostly because we had to carefully test and understand how everything fits together. It definitely helped us appreciate how real OS schedulers work.

2. Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Yes, working on priority donation gave us the most insight. It really made us think about how dependencies between threads can cause priority inversion and how the OS needs to handle it smartly. It was interesting to see how a relatively simple data structure like a lock can have complex interactions when multiple threads are involved. We also gained a better understanding of how thread state transitions are managed at a low level.

3. Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

One thing that would've helped us early on is a brief visual or diagram showing how priority donation flows through nested locks would have been super helpful. It took us a while to understand how deep donation works and how to manage it cleanly. Even a small illustration or animation could really solidify the concept for new students.
The guidance was mostly helpful, but a bit more clarity on when to disable interrupt especially during priority donation would've been useful. We were unsure at times, which led to some confusion and extra complexity. A quick guide on when to disable interrupts in these cases would help avoid issues like race conditions or deadlocks.

Team Member contribution on PA1:

Team member	Name	Test cases worked on
1	Indumathi Thamizharasan	**alarm-single** **alarm-multiple** **alarm-simultaneous** **alarm-priority** **alarm-zero** **alarm-negative** **priority-change** **priority-fifo** **priority-preempt**
2	Rishitha Saravanan Priya	**priority-donate-one** **priority-donate-multiple** **priority-donate-multiple2** **priority-donate-nest** **priority-donate-sema** **priority-donate-lower** **priority-sema** **priority-condvar** **priority-donate-chain**
3	Aishwarya Eshwar	**mlfqs-load-1** **mlfqs-load-60** **mlfqs-load-avg** **mlfqs-recent-1** **mlfqs-fair-2** **mlfqs-fair-20** **mlfqs-nice-2** **mlfqs-nice-10** **mlfqs-block**